Final Exam A: Computer Architecture, 2022 Spring

Student Name: _____ **Student AU ID:** _____

Instructions:

1. **Be ready.** Please read the instructions in full
2. **Be strategic.** Exam points are graded based on 100 points - we will later translate into a 7 step scale. Try to get as many points as you can. If you are stuck, try to move to another question and collect points.
3. **Be careful.** Please read the questions carefully before responding
4. **Be thorough.** Show us your procedure. No procedure and perfect result has few points. Good procedure with minor mistakes will get most points in a question.
5. **Be smart.** Do not copy text from the Internet. We will find out and there are consequences to it.
6. **Be good.** Please provide your name and AU ID on top. You will need the AU ID in some questions and you will get 1 point for writing it.

Assembly instructions help

`movq A B`	B = A
`salq A B` `sarq A B`	B = B << A B = B >> A
`mulq A B`	B = A * B
`divq A B`	B = B / A
`subq A B`	B = B - A
`cmpq A B`	Compare A to B (used before conditional expression)
`je [label]` `jne, jl, jle, jg, jge`	Jump to label if the previous line evaluates to “equal” Other variants (in the same order as listed): not equal, less, less or equal, greater, greater or equal
`leaq Src Dest`	Compute the memory address expression Src and put the result to Dest
`pushq Src`	Push the value at Src to the top of the stack
`popq Dest`	Read the value from the top of the stack and copy it to Dest

Theoretical Questions:

1) How do you calculate the number of bits needed to represent a positive integer?

Represent the following integers in binary: 55, 30, 242

Calculate: 55 & 30, 242 ^ 30

(4 points)

If n is the integer, then \ceil(n) bits

55 -> 00110111
30 -> 00011110
242-> 11110010

55 & 30 (bit by bit and) --> 00010110
242 ^30 (bit by bit XOR) --> 11101100

2) Briefly describe how fractional numbers are represented in binary. What is the binary representation of 342.3125? Can you represent 342.3 with 4 bits for the decimals? (4 points)

x_k x_{k-1} ... x_0 . x_{-1} x_{-2} x_{-3} ....x_{-p}

Converts to decimal: \sum_{i=-p}^{k} x_i 2^i

342.3125 --> 101010110.0101

Cannot represent 342.3 with 4 bits for the decimals

3) What is the purpose of the Jump instruction? (3 points)

The jump instruction allows you to go to a different address in the program execution (both forward and back)

4) What is the benefit of pipelining? What is the main limitation of pipelining ? ~~What are the two types of hazards when talking pipelining and why do they happen~~? (4 points)

The benefit of pipelining is that if we have multiple series of instructions going through the same pipeline, then when the first instruction series leaves the first stage we can start the next instruction series execution and not have to wait for the previous series to finish completely.

When we have nonuniform partitioning, meaning not all stages in the pipeline takes the same time, will we always be limited by the slowest stage.
Bonus: Though it is beneficial to have stages partitioned into equal execution times, it is sometimes difficult to do so and often impossible.

5) When a HDD needs to read an address, what takes up most of the time and what is relatively inexpensive? (4 points)

Read time = Seek time (moving the head) + rotational latency (rotate the platter) + transfer time of reading the actual bits.
First two are slow, reading bits is inexpensive.
Seek time is usually the most expensive one, though

6) What is a Page Table and a Page Fault? How are Page Faults handled? What is Memory Thrashing and why do poorly written programs cause it? (4 points)

Page Table: a big lookup table that maps virtual pages to physical pages.
Page fault: the requested memory address is on a page that's not in memory.
Handling: Exception is raised, the page is pulled from the disk and the violating instruction is repeated (guaranteed page hit this time).

Memory Thrashing is when every memory read results in a page fault. Programs with bad locality may cause it.

7) What do the following two operations do? What is their difference? Which one would correspond to the C code: *p = var ? (4 points)

- `movq %rax,(%rdx)`
- `movq (%rax),%rdx`

- Moves contents of register %rax to the address indicated by register %rbx
- Moves the content at address given by %rax to register %rdx
- First case corresponds to *p = var

8) What are the two types of hazards when talking pipelining? why do they happen? (4 points)

The first is data hazards, can happen if we do not write to registres at the appropriate time doing execution or read wrong values from registres.

The second is Control hazards, they happen when the
CPU cannot predict the address of the next instruction. Exmple: conditional jumps

9) What is a semaphore? What problem does it solve and how? (2 points)

A global integer used as a synchronization primitive to lock a variable when a thread is changing it.
It makes sure other threads wait while it's locked (value of integer is equal to zero).
Usually implemented with two functions P(s) and V(s) to request and release resources and update the s value (the integer)

Practical Questions (11 points each):

1) Consider the following C code function

```
#include <stdio.h>
int operat(unsigned int x, unsigned int y)
{
   unsigned int t = x << 4;
   t = t - x;
   y = y >> 4
   return t+y;
}
```

- What is the final result in terms of 'x' and 'y'? *Hint: you can express it as a mathematical function. Think of the interpretation of << and >>*
- For values of x is your function valid? *Hint: consider UINT_MAX as the maximum representable unsigned integer.*
- If x = 4 and y = 192, what does the function return?

t = 16 x - x
Returned value = 15x + floor(y/16)

Basic answer (ok for most of the grade): x <= floor(UNIT_MAX/16) or x < UINT_MAX/16 to avoid overflow in the first step

More advanced: 15* x + y /16 <= floor(UINT_MAX/2) and x <= floor(UNIT_MAX/32) (avoid overflow given signed integer output and unsigned integer input)
—> x < floor(UNIT_MAX/2)/15 - y/(15*16). and x < UNIT_MAX/32

15*4 + (192/16) = 72

2) The C function below calculates the product of two square matrices. Analyze the code and answer the following questions:

- How many memory reads and writes are performed in the loop that dominates the function's speed? a) Two memory reads, One write
- What is the average cache miss in each iteration of this loop?
- Is it possible to make the code faster? How, and what's the average cache miss rate of the improved configuration?

Matrix multiplication function:

```
void matrix_mul(float *a, float *b, float *c, int n){

  float sum = 0;
  int i, j, k;
  for (i=0; i<n; i++)  {
    for (j=0; j<n; j++) {
      sum = 0.0;
      for (k=0; k<n; k++)
        sum += a[i][k] * b[k][j];
      c[i][j] = sum;
    }
  }
}
```

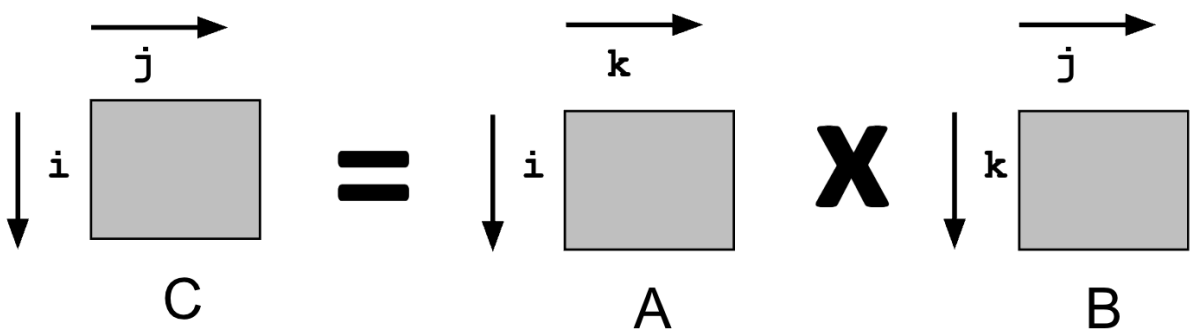
c. Improved case (trick seen in lectures)

```
for (k=0; k<n; k++) {

   for (i=0; i<n; i++) {
      r = a[i][k];
      for (j=0; j<n; j++)
         c[i][j] += r * b[k][j];
   }
}
```

- Cache miss ratio per variable: A is not involved in loop (so miss ratio is 0)
- c and b follow stride-1 pattern

miss ratio for each is 4/64 = 1/16
(that is miss once, hit 15 times after that)

Hint: Matrix indexes are given by



Assumptions:

- `sizeof(double) = 4`
- A cache block is 64 bytes

b.
Given assumptions (4 byte data type, 64 byte cache block), miss ratio per inner loop iteration for each variable/array:

Read a	Read b	write sum	write C
4/64.	1.	0.	0
stride-1 ,	jumps > 64 bytes ,	reused constantly, not in inner loop	

The matrix dimension (`n`) is much higher than a cache line size

3) Given the following C code and resulting assembly code:

```
void switch_values_1(long* xp, long* yp, long* zp)
{
    long x = *xp;
    long y = *yp;
    long z = *zp;

    *yp = x;
    *zp = y;
    *xp = z;
}
```

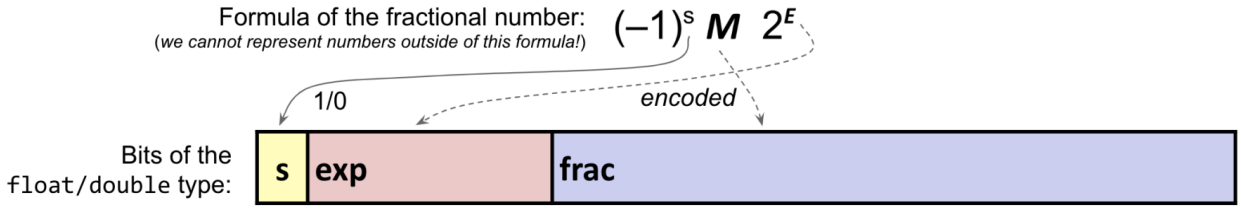
```
# void switch_values_1(long* xp, long* yp, long* zp)
# xp: %rdi, yp: %rsi, zp: %rdx

switch_values_1:
    movq    (%rdi), %rax
    movq    (%rsi), %rbx
    movq    (%rdx), %rcx

    movq    %rax, (%rsi)
    movq    %rbx, (%rdx)
    movq    %rcx, (%rdi)
    ret
```

How would the assembly code change if the code changes to a slightly different function?

```
void change_values_1(long* xp, long* yp, long z)
{
    long x = *xp;
    long y = *yp;

    *yp = x * z;
    *xp = y + z;
}
```

```
change_values_1:

movq   (%rdi), %rax
movq   (%rsi), %rbx

mulq   %rdx, %rax
movq   %rax, (%rsi)
addq   %rbx, %rdx
movq   %rdx, (%rdi)
```

4) Consider the following 8-bit IEEE-format floating point representation where each number has 3 exponent bits and 4 fraction bits.



If a value is represented with the following 11011100 bit sequence, answer the following:

a. Calculate the exponent bias
b. Calculate the value
c. Round the number to have 3 fraction bits using the round-to-even mode

a. Bias: e = 2^{3-1} -1 = 3

b.

exponent: (101)b —> 5 in decimal
E = 5 - 3 = 2
M = 1.1100
sign= (-1)^1 = -1

value = - (1+ 1/2 + 1/4) x 2^2 = - (4 +2 +1) = -7 in decimal

c.
Only least significant bit is removed and it is valued at zero.
Sequence becomes: 1101110, which is still -7 in decimal

5) Describe the operation of the following HCL code functions, i.e., what does it achieve. As in class, assume that capitalized letters are words (e.g., integers of n bits). Otherwise, they represent booleans.

```
int function1 = [  A > B && A > C : A;
                   B > A && B > C : B;
                   1              : C;
                 ];
```

Main purpose: Selects the maximum of the 3 words
Execution is 'lazy', meaning that first line of conditions are evaluated, then the second ONLY IF the previous are not true (so the first line must be false as a whole). ONLY IF the second is also not true, then the third line is evaluated
This is the reason the last condition is '1': it is a default case if the other conditions are not true

```
int function2 = [  !s1&&!s0: D0;
                   !s1     : D1;
                   1       : D2;
];
```

Same logic as before to read the conditions

In this case, it has two lines/booleans (s1 and s0) that select the output word

D0 input is selected as output, if s1 = 0 and s0 = 0
D1 input is selected as output, if s1 = 0 and s0 = 1
D2 input is selected as output in all other cases, meaning: s1 = 1 and s0 = 0 OR s1 = 1 and s0 = 1

6) Consider a 2-way associative cache with the following properties:

Main memory size: 64 bytes

Cache block size: 4 bytes

E = 2 block/set

- Describe the cache structure and the bits of each address used for selecting the set the tag.
- The code reads bytes through a one dimensional `char` array of 32 elements (i.e., 32 bytes). The byte array starts in address 0x000. It reads in a stride-one pattern. Which address reads cause a cache miss? What is the cache miss ratio?
- Consider the same array as before, but first read the even addresses (0, 2, 4, …, 30) followed by the odd ones (1,3,5,…,31). How does the cache miss ratio change?

1) Main memory of 64 bytes implies 6 bit addresses
Cache block size of 4 bytes implies the 2 least significant bits of the address are the offset
Two-way associative cache means that there are two sets and thus 1 bit to tell them apart

Thus the structure is: ADDRESS [6 bits] = [TAG [3 bits] / SET [1 bit] / OFFSET [2 bits]]

In terms of structure: 2 sets each with 2 blocks of 4 bytes each.
Total cache size for storage of block data: 2 sets x 2 blocks/set x 4 bytes/block = 16 bytes

2) Stride 1 pattern mean address 0, then address 1, then address 2, etc
Address 0x00 causes a cold miss (1st set, 1st block). Address 0x01, 0x02, 0x03 are hits as they are already in cache
Address 0x04 causes a cold miss (2nd set, 1st block). Address 0x05, 0x06, 0x07 are hits as they are already in cache
Address 0x08 causes a cold miss (1st set, use 2nd block). Address 0x09 - 0x0B are hits
Address 0x0C causes a cold miss (2nd set, use 2nd block). Address 0x0D - 0x0F are hits
—> Assume least use replacement strategy <—
Address 0x10 causes a conflict miss (1st set, use 1st block). Address 0x11 - 0x13 are hits
Address 0x14 causes a conflict miss (2nd set, use 1st block). Address 0x15 - 0x17 are hits
Address 0x18 causes a conflict miss (1st set, use 2nd block). Address 0x19 - 0x1B are hits
Address 0x1C causes a conflict miss (2nd set, use 2nd block). Address 0x1D - 0x1F are hits

1/4 reads cause a miss for this cache, this read pattern, the chosen replacement strategy, and this 32-byte array

3) Basically, it operates as two consecutive rounds of the previous read, but skipping first the odds and then the even addresses
By the time the first 16 bytes with even addresses are read, the cache will not contain useful information for reading the odd ones

Thus, the cache miss ratio is 1/2. Miss once and hit once.
Example: reads address 0x00 (miss) and get one hit for address (0x02)